

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

– HoChiMinh, October 2026 –

Table of Contents

I. Record of Changes..... 3

II. Software Design Document..... 4

1. System Design..... 4

1.1 System Architecture..... 4

1.2 Package Diagram..... 4

2. Database Design..... 4

3. Detailed Design..... 5

3.1 <Feature/Function Name1>..... 5

3.2 <Feature/Function Name2>..... 6

I. Record of Changes

Date	A* M, D	In charge	Change Description
29/10/2026	M	Võ Minh Luân	1.1 System Architecture: redrew Figure 1 (layout, connectors, numbered flow steps and key) for readability; components and flows unchanged.
29/10/2026	M	Nguyễn Xuân Lộc	2.1 Overview: corrected the migration range to V1–V61.
29/10/2026	M	Trần Võ Quang Duy	3.2 Activity Diagrams: aligned the section titles with the workflow table (WF-01 to WF-05).

*A - Added M - Modified D - Deleted

II. Software Design Document

Every diagram of this document is a draw.io file in the repository (docs/diagrams/*.drawio), drawn with draw.io's UML shapes and exported to the pictures below by draw.io itself (node docs/tools/drawio.mjs export). The ERDs use Chen's notation: entities as rectangles, attributes as ovals (the key underlined), relationships as diamonds with the cardinalities 1 and N; they and the table lists are generated from the migrated schema (docs/tools/gen-db-docs.mjs), so they cannot drift from the database script.

1. System Design

1.1 System Architecture

CampusStay FPT is one Spring Boot API (a modular monolith with one database) used by two clients: a React web app for the four roles and a Flutter mobile app for students (built for Android). The web app is a static build served by Vercel; the API runs in a Docker container behind Caddy on an Azure virtual machine and stores its data in a managed PostgreSQL server. Four external services are used: Google Identity (sign-in), Firebase Cloud Messaging (push), Gmail SMTP (email) and a VietQR image service (the QR picture of a transfer). Money moves outside the system: a student transfers to the dormitory's bank account and a Manager confirms the student's claim; there is no bank API.

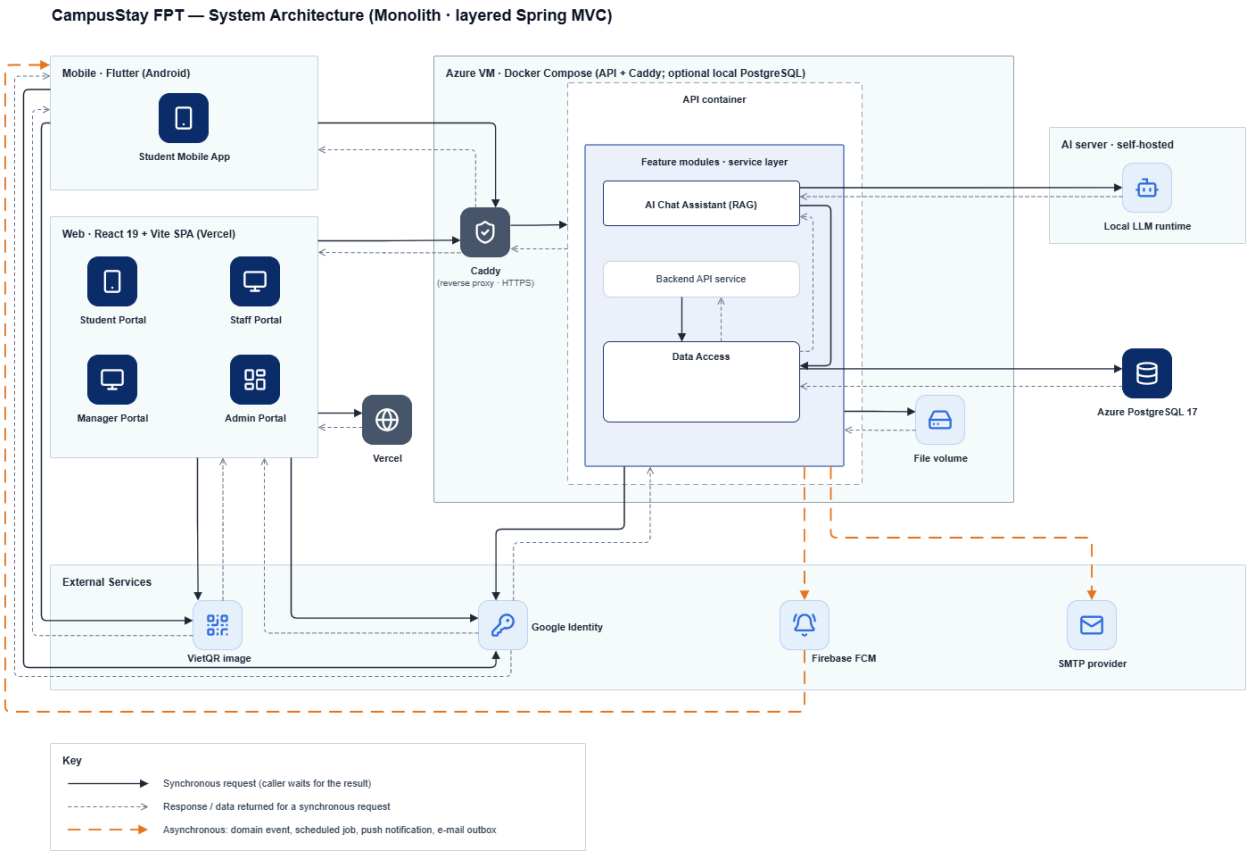


Figure 1: System overview: components, external services and the main data flows

Component	Responsibility	Technology
Web app	Student portal (phone-first) and the Staff, Manager and Admin back office; calls only /api with a bearer token; Vietnamese and English	React 19, React Router 8, Vite 8, Tailwind CSS 4
Mobile app (Flutter, Android)	Students: Home, My Room (door code, check-in pass), Payments (VietQR, claim), Activity (notifications, requests); push notifications	Flutter 3.47 (Dart 3.13), http, Firebase Messaging
Caddy	Reverse proxy on the VM: HTTPS with Let's Encrypt certificates, forwards /api to the API container	Caddy 2
CampusStay API	REST API (/api): security filter (bearer session, role rules), controllers, services with the business rules and transactions, repositories; scheduled jobs; domain events; email outbox; push after commit	Spring Boot 4.1.1, Java 21
PostgreSQL	One database: 98 tables, 32 report views; constraints and partial unique indexes carry the critical rules (one Allocation per Bed per Term, one payment per Invoice)	Azure Database for PostgreSQL 17 (Flexible Server)
File volume	Uploaded photos, receipts and evidence files, served through short-lived signed links	Docker volume on the VM
Google Identity Services	Issues ID tokens of @fpt.edu.vn accounts; the API verifies them with Google's public keys	OAuth 2.0 / OpenID Connect
Firebase Cloud Messaging	Delivers push messages to the students' Android devices	FCM HTTP v1
Gmail SMTP	Sends emails queued in the outbox (reset links, invitations, warnings, campaigns)	SMTP over TLS
VietQR image service	Draws the QR picture of a transfer (bank, account, amount, transfer content)	HTTPS image URL

Main architectural decisions (full list: Design Brief section 3 and docs/00_Project_Brief.md Decision Log):

ID	Decision	Reason
ADR-01	One deployable and one database (monolith)	Allocation and payment need ACID transactions across features; three clients share one API; a small team and a 16-week schedule.
ADR-17	Layered packages: controller → service → repository, one sub-package per feature	The standard Spring layout is easy to navigate; LayeringTest checks that calls go one way.

ADR-07	Opaque server-side sessions (bearer token, only its SHA-256 stored)	Instant revocation at sign-out, password change or disabled account; no key management.
ADR-06	Students choose Room Type and Room Class, never a Bed; a batch engine assigns	Gender and Block rules and fairness are guaranteed by the engine; no race for the same Bed.
ADR-22	Payments are claims confirmed by Managers	No bank API or payment-gateway contract is available; the claim stops the deadline so a slow confirmation never costs a place.
ADR-16	Web on Vercel; API on one VM with Docker Compose and Caddy; GitHub Actions deploys	A VM never sleeps, so jobs, the email outbox, SMTP and the file volume work; Vercel cannot run a long-lived Java process.

1.2 Package Diagram

No	Package	Description

No	Package	Description

1.3 Technology Declaration

1.3.1 Development technologies

Layer	Technology and version	Notes
Front-end (web)	React 19.3, React Router 8.4, Vite 8.3, Tailwind CSS 4.3, lucide-react 1.59, qrcode-generator 1.4.4, Inter font	JavaScript (JSX); built to static files
Back-end	Java 21 (Temurin), Spring Boot 4.1.1 (Web MVC, Validation, Security, OAuth2 JOSE, Data JPA, Flyway, Mail, Actuator), Maven	One deployable campusstay-backend
Mobile	Flutter 3.47.8 (Dart 3.13, Material 3), http 1.6, shared_preferences 2.5, qr_flutter 4.1 (QR), firebase_core 5.1 + firebase_messaging 17.0, google_sign_in 7.2 (Credential Manager); Android host: Android Gradle Plugin 9.1	compileSdk/targetSdk 36, minSdk 26 (Android 8.0)
Database	PostgreSQL 17 (production: Azure Database for PostgreSQL Flexible Server; tests: Testcontainers postgres:17-alpine; development: local PostgreSQL), Flyway migrations V1–V61	Schema owned by Flyway; the migrations are the database script
Testing	JUnit 5, AssertJ, Spring Boot Test, Testcontainers 2.0, Maven Surefire (unit) and Failsafe (integration)	235 automated tests (section 3 of Report 5)
Packaging and runtime	Docker (API image built by GitHub Actions), Docker Compose, Caddy 2	Images in GitHub Container Registry

1.3.2 Third-party services

Service	Purpose	Data sent	Configured by
Google Identity Services	Authentication: sign in with an FPT Google account (web popup or redirect code flow, Android Credential	The authorization code or ID token; the API receives email, name, Google subject and hosted domain	GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET (server), VITE_GOOGLE_CLIENT_ID (web)

	Manager)		
Firebase Cloud Messaging	Push notifications to the Android app	Device token, notification title, body and link (never a door code or an amount owed by someone else)	FIREBASE_CREDENTIALS_FILE (service account on the server); google-services.json in the app (not in Git)
Gmail SMTP	Email: password reset, invitations, payment and Utility warnings, Admin campaigns	Recipient address, subject, HTML body; on the demo server emails of fictional accounts go to the team mailbox	SPRING_MAIL_* (App Password), MAIL_REDIRECT_TO
VietQR image service (qr.sepay.vn)	Payment: the QR picture shown on the payment page	Bank, account number, account name, amount, transfer content CSTAY… (no personal data, no secret)	PAYMENT_*
Vercel	Hosting of the web app (static build, CDN, HTTPS)	The built web files; visitors' requests	Vercel project campusstay-fpt
Microsoft Azure (Azure for Students)	Hosting: VM for the API, managed PostgreSQL	All business data (fictional demo data)	infra/prod/, runbook appendix A
GitHub, GitHub Actions, GHCR	Source code, CI/CD, container images	Source code, built images	.github/workflows/backend.yml

No AI API is called by the product. Secrets live only in the server's /opt/campusstay/.env and the hosting settings, never in Git (infra/prod/.env.example lists the names).

1.3.3 Source code and DevOps

Topic	**Practice**
Platform	GitHub, one monorepo tisou47/CampusStay-FPT (ADR-05): backend/, web/, mobile/, docs/, infra/, .github/workflows/
Branches	main is production. Work is done on a branch per change (feat/…, fix/…, docs/…); until Review 2 each phase of the Change Specification v2 is built on feat/v2-phase<n> and merged when the phase is complete, its checks pass and the demo works. main is updated only by fast-forward after the checks pass.
Commits	Conventional format with the feature code: feat(web): …, fix(bil): …, docs: ….
Checks before merging	mvn -o verify in backend/ for any backend change (unit, integration and architecture tests; Testcontainers needs Docker); npm run build in web/ for

	any web change; flutter analyze, flutter test and flutter build apk in mobile/ for any mobile change. Never merge with a failing check.
CI/CD (backend)	GitHub Actions backend.yml on every push to main touching backend/** or the production stack: job verify (mvn verify on Ubuntu with Temurin 21), job image (Docker build, push to GHCR tagged with the commit SHA and latest), job deploy (SSH to the VM, docker compose pull and up -d api, readiness check GET /api/actuator/health = UP). Pull requests run verify only.
CD (web)	Vercel builds production on every push to main and a preview deployment for other branches when web/ changed.
Database changes	Only new Flyway migrations (V<next>__….sql); an applied migration is never edited; migrations run when the new image starts, after passing mvn verify.

1.3.4 Deployment environments

Environment	Where	How it is deployed	Data	Rollback

Before the final defense the production server is also the demo (staging) environment declared for Defense 1.1: the council's scenarios run on it with data loaded by the demo tool, which is a script in the repository (backend/src/main/resources/db/demo/), so every run is repeatable.

1.3.5 AI tools

Tool	Model	Stages where it is used	Rules followed
Claude Code (Anthropic), in IntelliJ IDEA, VS Code and the Claude desktop app	Claude Opus family (Claude Opus 5.5 at the time of writing)	Specification: drafting the Change Specification v2, SRS tables and business rules from the team's decisions. Design: ADRs, the draw.io diagrams (laid out by generators in docs/tools/), database migrations. Code: backend features, web screens, Android screens, generators in docs/tools/. Testing: integration tests and the test catalog. Review: code review of each	Every AI output is reviewed and tested (mvn verify, npm run build) before it reaches main; no real personal data, passwords, API keys or secrets are given to the tool (fictional seed data; secrets stay in the server .env and a local folder outside the repository); each member answers for the parts they own (Report 2, ownership).

		change and the BM02 self-review with the standard prompt.	

2. Database Design

2.1 Overview

The database script is the set of Flyway migrations `backend/src/main/resources/db/migration/V1_…sql` to `V59_…sql`, applied automatically when the API starts; the base data are `db/seed/R_dev_seed.sql`. The logical ERDs and table lists below are generated by `docs/tools/gen-db-docs.mjs` from `information_schema` of a database migrated to V61 (98 tables), so every table and key in the ERD exists in the script and every table of the script is in an ERD. The ERDs are drawn in Chen's notation: a rectangle per table, ovals for its key (underlined) and main business attributes, and a diamond per foreign key, named after the column, with 1 on the referenced side and N on the referencing side (1 when the key is also the whole primary key). Tables of another area appear as dashed rectangles. The complete columns, types, checks and indexes are in the migrations.

Conventions: tables are prefixed with their feature (`auth_`, `cfg_`, `inv_`, `app_`, `asg_`, `bil_`, `req_`, `mnt_`, `ntf_`, `shared_`); keys are `BIGINT` identities; money is `BIGINT` VND; times are `TIMESTAMPTZ` in UTC; statuses are upper-case text with a `CHECK`; mutable rows carry a `version` for optimistic locking; JSONB only for snapshots (Invoice lines, contact snapshot, matching snapshot). Reports read 32 views `rpt_v_*`, each defined by the feature that owns the tables.

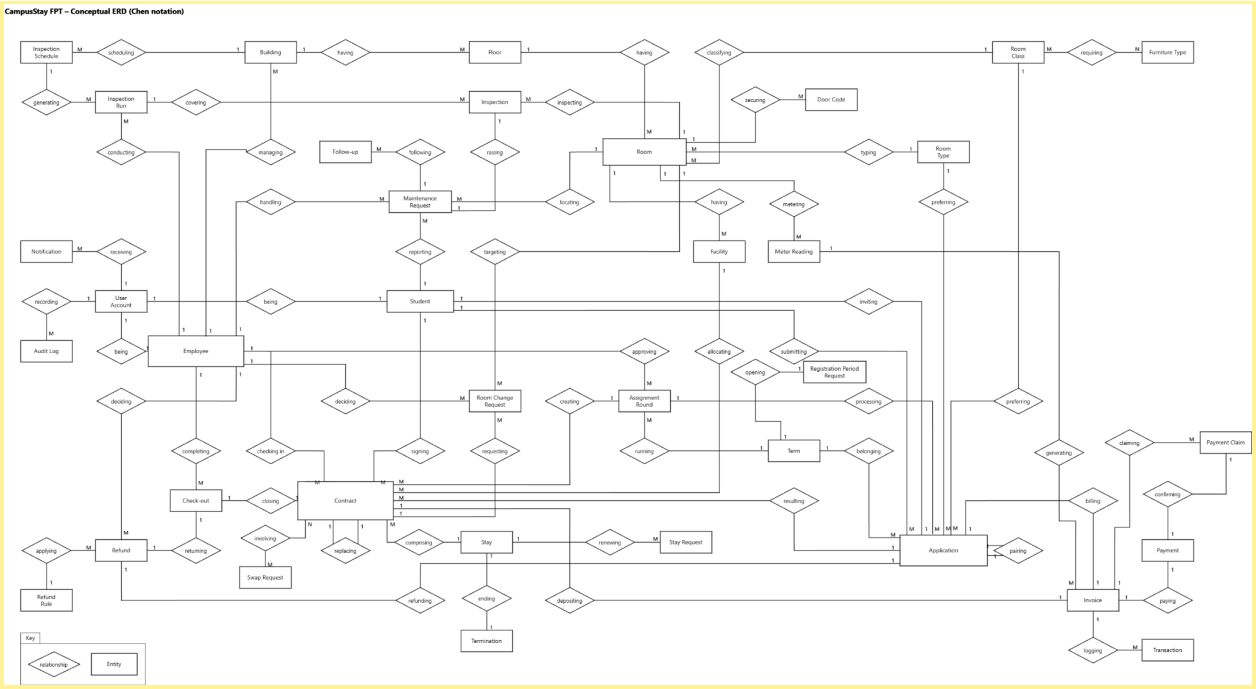


Figure 2: Conceptual ERD of the main entities in Chen's notation; the logical ERD of each area (all 98 tables) follows in sections 2.2 – 2.11

Rules guaranteed by the database (the safety net behind the services' checks):

Rule	Constraint
One active Allocation per Bed per Term; one per student per Term	Partial unique indexes ux_alloc_bed_term, ux_alloc_student_term; Beds locked with SELECT … FOR UPDATE when they change hands
One live Application per student per Term	ux_app_one_live_per_term
Published Terms never overlap	Exclusion constraint ex_term_no_overlap
One live assignment run per Term	ux_asg_one_live_run
One open claim and one payment per Invoice; one refund per Invoice	ux_claim_open, ux_payment_one_confirmed, ux_refund_one_per_invoice
An Invoice's money never changes; Paid and Void are final	Trigger tr_invoice_fixed
One next-Term fee per Allocation	ux_invoice_one_term_fee
One current and one waiting door code per Room	ux_door_code_active, ux_door_code_proposed
One open registration-period request per Term	ux_term_request_open
Each Floor number is unique within a Building; each Bed label is unique within a Room	ux_floor, ux_bed
A Room's Building matches the Building of its Floor	Trigger trg_inv_room_building
A Room out of service has a date and a reason	ck_room_oos
One open door-code change task per Room	ux_door_task_open
One planned move per furniture item	ux_furniture_planned_move
A published Term has every window set (registration, results, deposit, check-in, check-out)	ck_term_published_complete
One registration-period request per Term unless it was rejected	ux_term_request_term
A roommate pairing only in invite mode; a rejected Application has a note	ck_app_partner, ck_app_rejected
One pending roommate invitation per inviter	ux_invite_one_pending_per_inviter
An assignment run proposes each Bed and each Application at most once; a waitlisted Application has no Bed	ux_proposal_bed, ux_proposal_app, ck_proposal_waitlist

Round 2: one pending offer per Application; each Bed is held by one option	ux_round2_one_pending, ux_round2_bed_held
One active Stay per student	ux_stay_active
One Check-out per Allocation; a completed Check-out has meter readings, the room condition and the key return	UNIQUE (allocation_id) on asg_check_out, ck_checkout_completed
One open Termination per Allocation	ux_termination_open
One pending late check-in per Allocation	ux_late_one_pending
A renewal moves to a different Term	ck_renewal_terms
One open Room Change Request per student per Term	ux_rc_one_open
A Swap is between two different Allocations; one open swap started by each Allocation and one active swap post per Allocation	ck_swap_distinct, ux_swap_one_open_a, ux_swap_post_one_active
One open Stay Request per student per Term	ux_stay_request_open
One open room-class change per Room per Term	ux_class_change_open
One deposit Invoice per Application	ux_invoice_one_deposit
A refund never exceeds the amount paid; a rejected or exception refund has a note	ck_refund_amount, ck_refund_note, ck_refund_exception_note
A bank transaction is recorded once and matches at most one Payment	ux_bank_txn, UNIQUE (bank_transaction_id) on bil_payment
One Meter Reading per Room per period; one period per Building per start date; readings never go down	ux_meter_reading, ux_meter_period, ck_meter_period, ck_meter_monotonic
One open utility warning per Allocation per level	ux_utility_warning_open
An Inspection Run checks each Room once	ux_inspection_room
One published version per checklist; version numbers never repeat	ux_checklist_one_published, ux_checklist_version
One open escalation per source	ux_escalation_open
An account email is unique, ignoring case	ux_auth_user_email on lower(email)

2.2 Accounts and access (AUTH)

No	Table	Description	Keys

2.3 Settings and Terms (CFG)

No	Table	Description	Keys

2.4 Inventory: Blocks, Rooms, Beds, furniture, door codes (INV)

No	Table	Description	Keys

2.5 Applications (APP)

No	Table	Description	Keys

2.6 Assignment and residence (ASG)

No	Table	Description	Keys

2.7 Billing and Utility (BIL)

No	Table	Description	Keys

2.8 Requests, routing and escalations (REQ)

No	Table	Description	Keys

2.9 Maintenance and inspections (MNT)

No	Table	Description	Keys

2.10 Notifications and announcements (NTF)

No	Table	Description	Keys

2.11 Shared: Audit Log and files

No	Table	Description	Keys

3. Detailed Design

3.1 State Machines of the Main Entities

Each diagram names, on every transition, what triggers it and the functional requirement (FR) of Report 3 that specifies it; transitions marked job are made by the scheduled jobs. The states are the values of the status column of the table named in the title.

3.1.1 Application (app_application.status)

The status DRAFT exists in the schema but the current flow never stores it: an Application is created when the student submits the wizard.

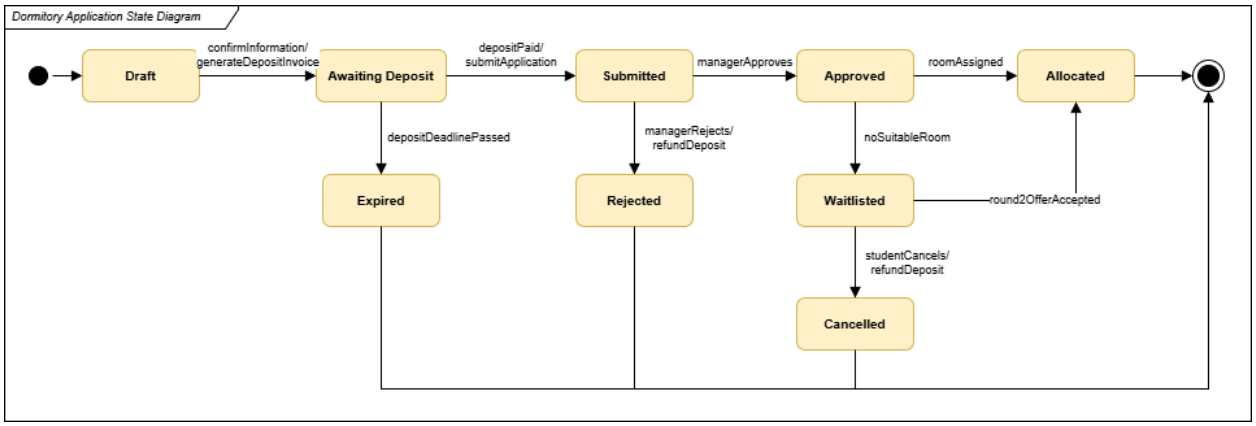


Figure 3: State machine: Application

3.1.2 Invoice (bil_invoice.status)

The status UPCOMING exists in the schema for reading only; Invoices are issued Unpaid.

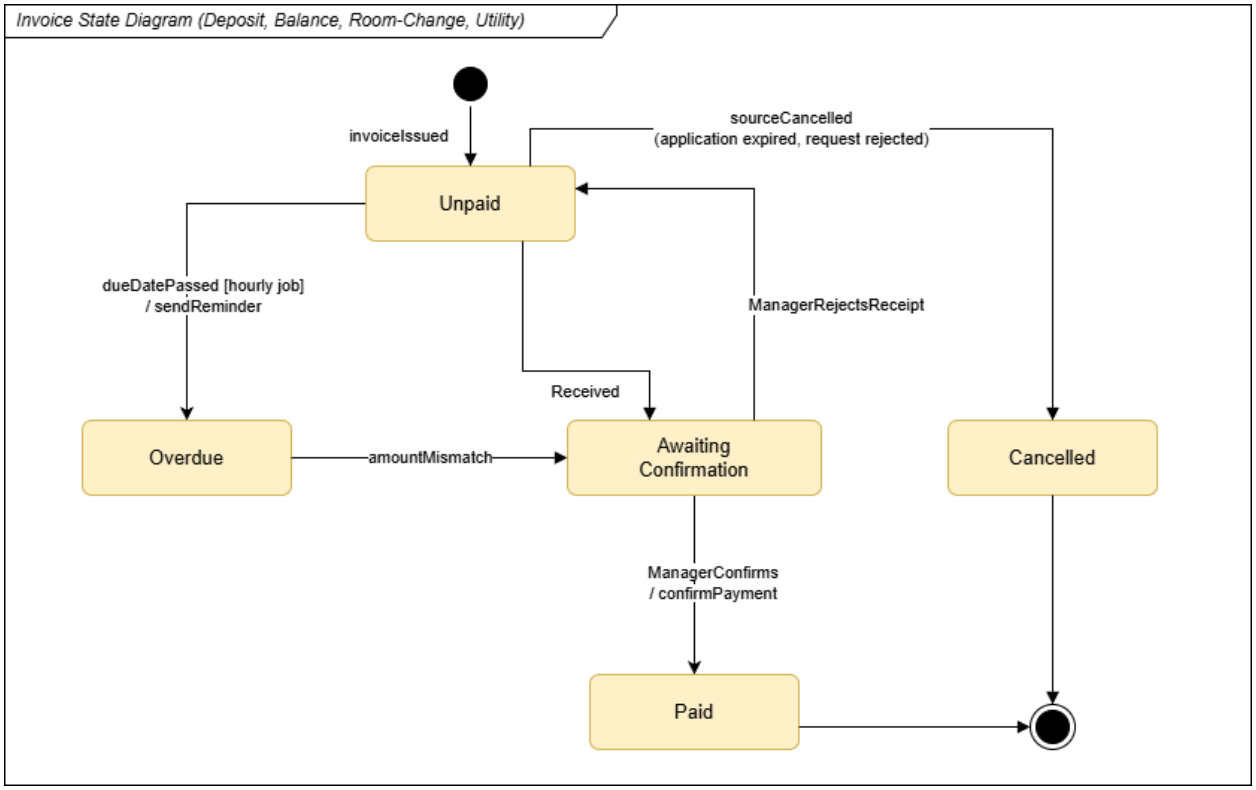


Figure 4: State machine: Invoice

3.1.3 Maintenance Request (mnt_request.status)

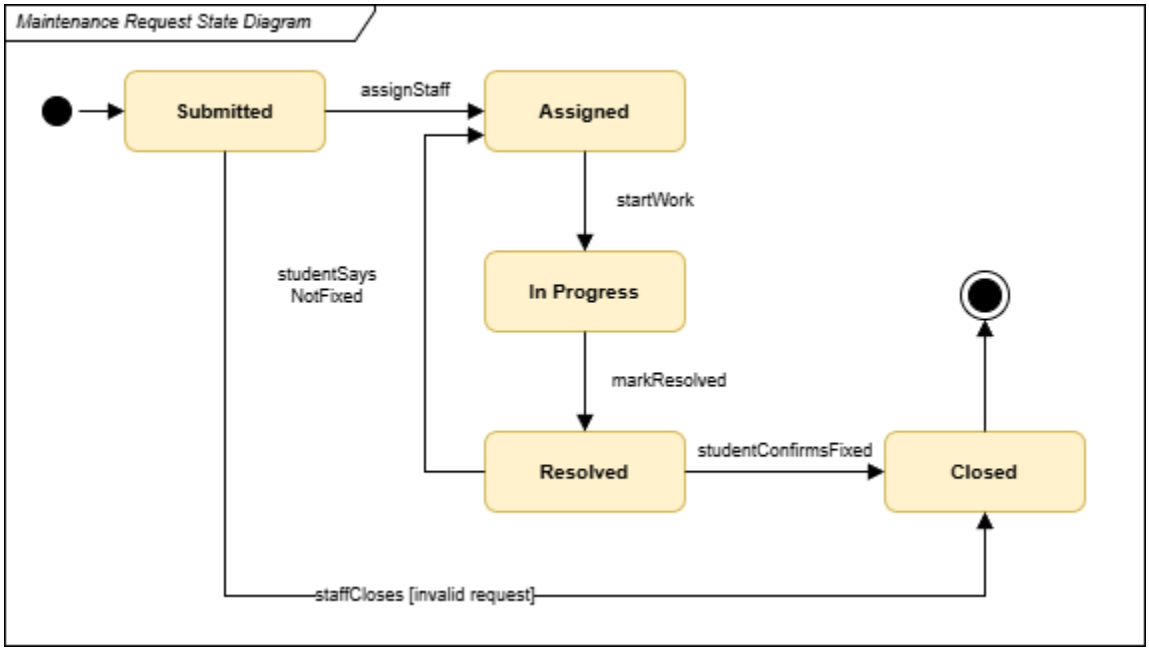


Figure 5: State machine: Maintenance Request

3.1.4 Next-Term requests

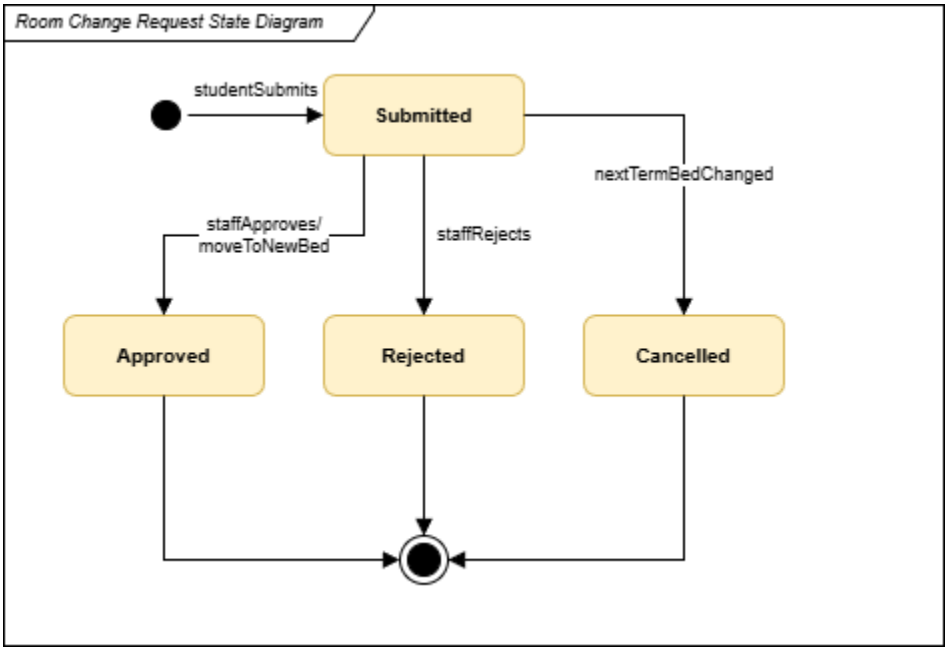


Figure 6: State machine: Room-Change Request (next Term)

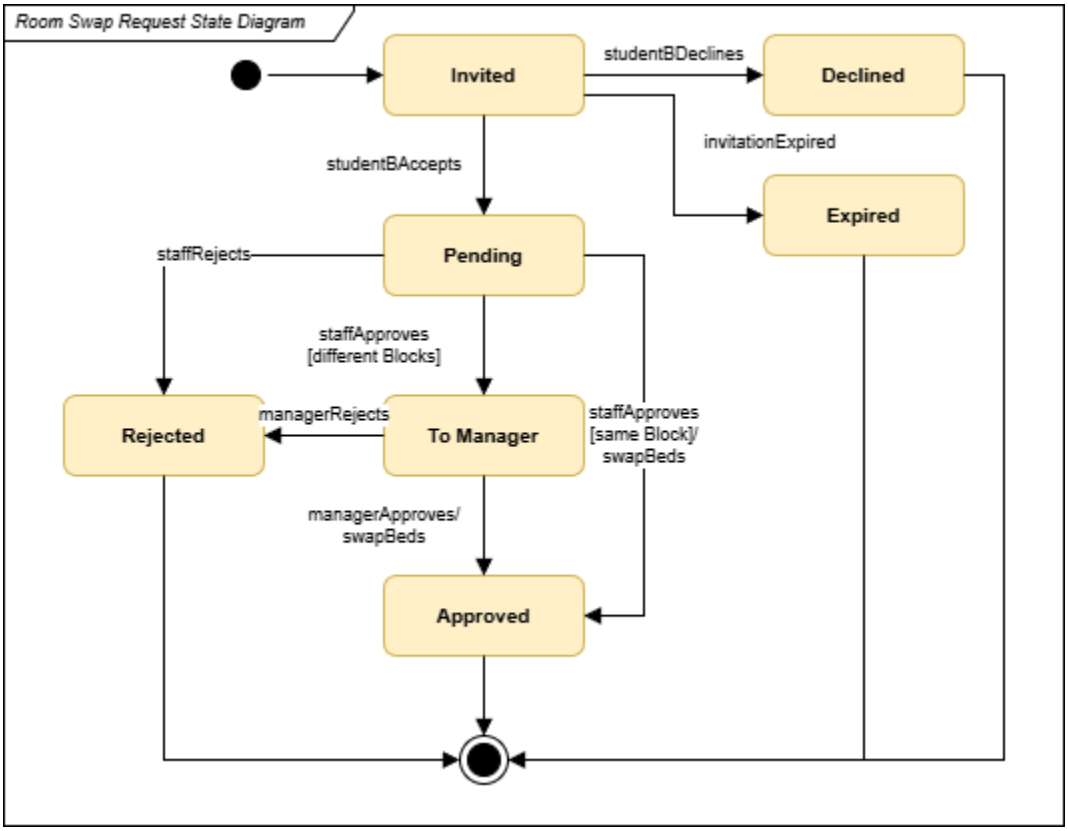


Figure 6.1: State machine: Room-Swap Request (next Term)

3.1.5 Late check-in, termination, door code and Round-2 offer

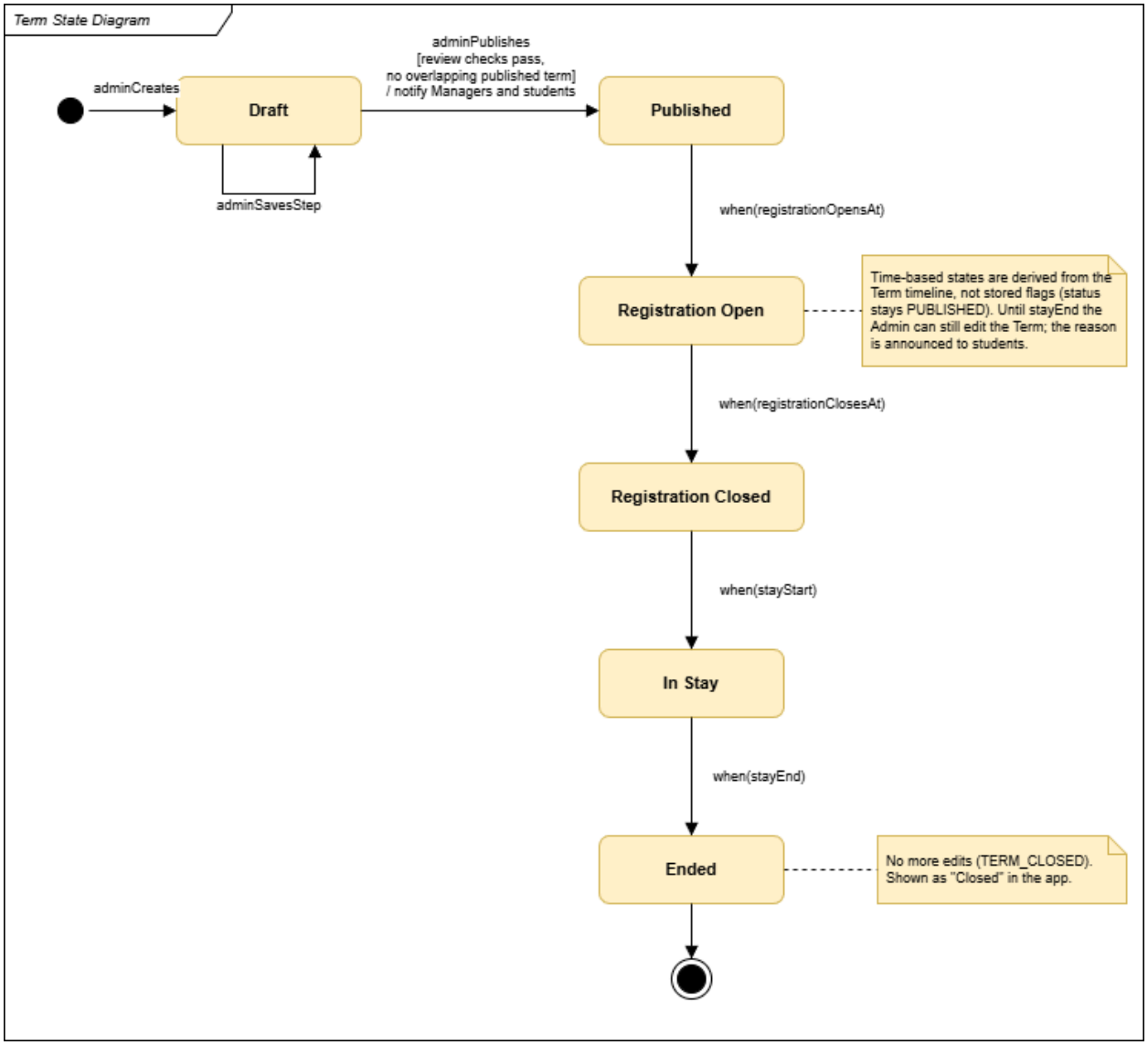


Figure 7: State machine: termination

3.2 Activity Diagrams of the Main Workflows

The five main workflows of the system (Design Brief Appendix A1). Lanes are the actors; the System lane holds what the API and its jobs do. Each workflow has one owner, its demo scenarios and its end-to-end test case (Report 5).

ID	Workflow	Actors	Owner
WF-01	Registration to residence	Student, Manager, System	Lê Thành Minh Trung

WF-02	Opening a registration period	Manager, Administrator, System, Student	Trần Võ Quang Duy
WF-03	Term transition of continuing residents	Student, Staff, Manager, System	Nguyễn Xuân Lộc
WF-04	Maintenance with routing and time limits	Student, Staff, Manager, System	Võ Minh Luân
WF-05	Monthly Utility, debt and termination	Staff, Student, Manager, System	Lê Thành Minh Trung

3.2.1 WF-01 Dormitory Registration

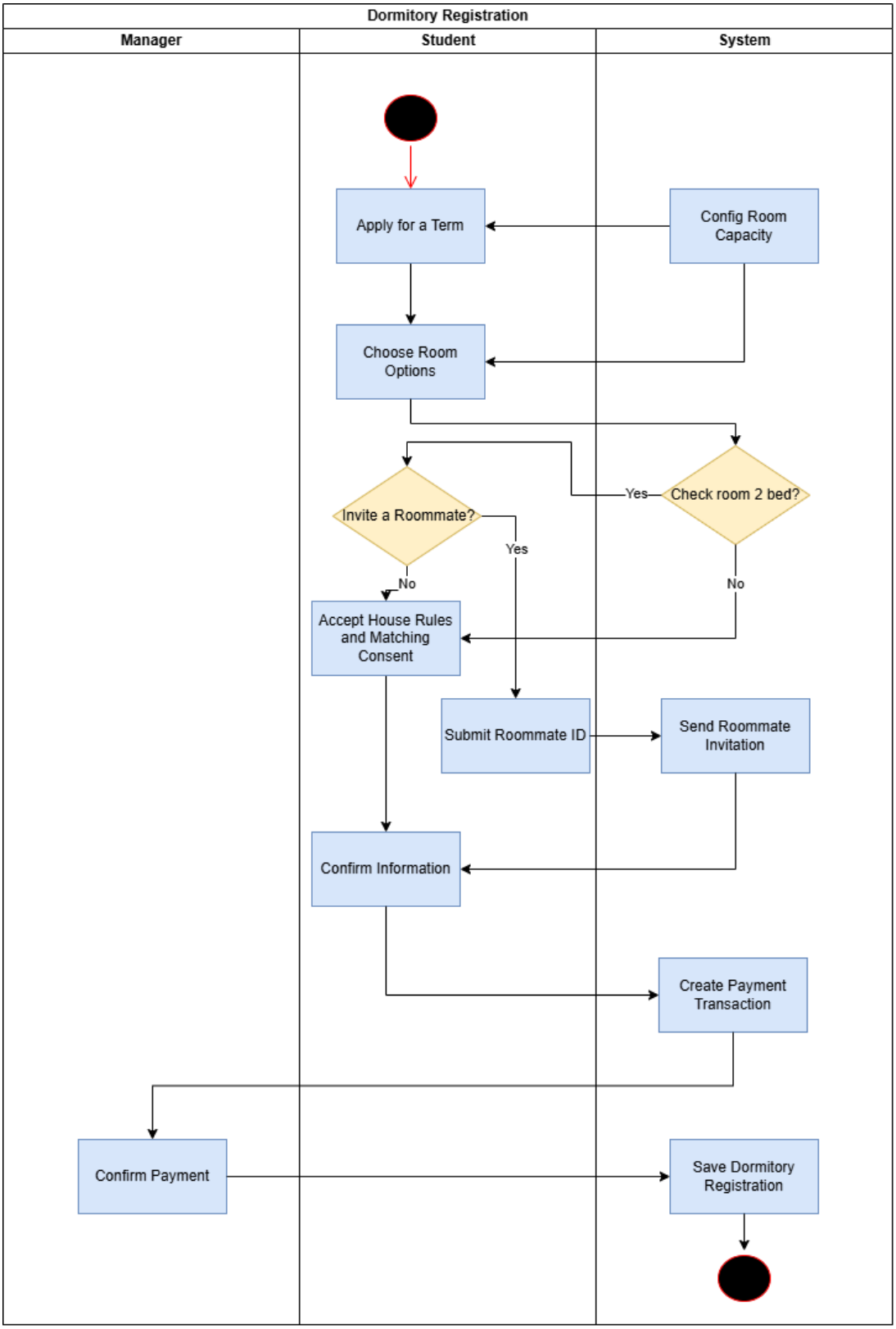


Figure 8: Activity diagram WF-01: Dormitory Registration

3.2.2 WF-02 Room & Bed Assignment

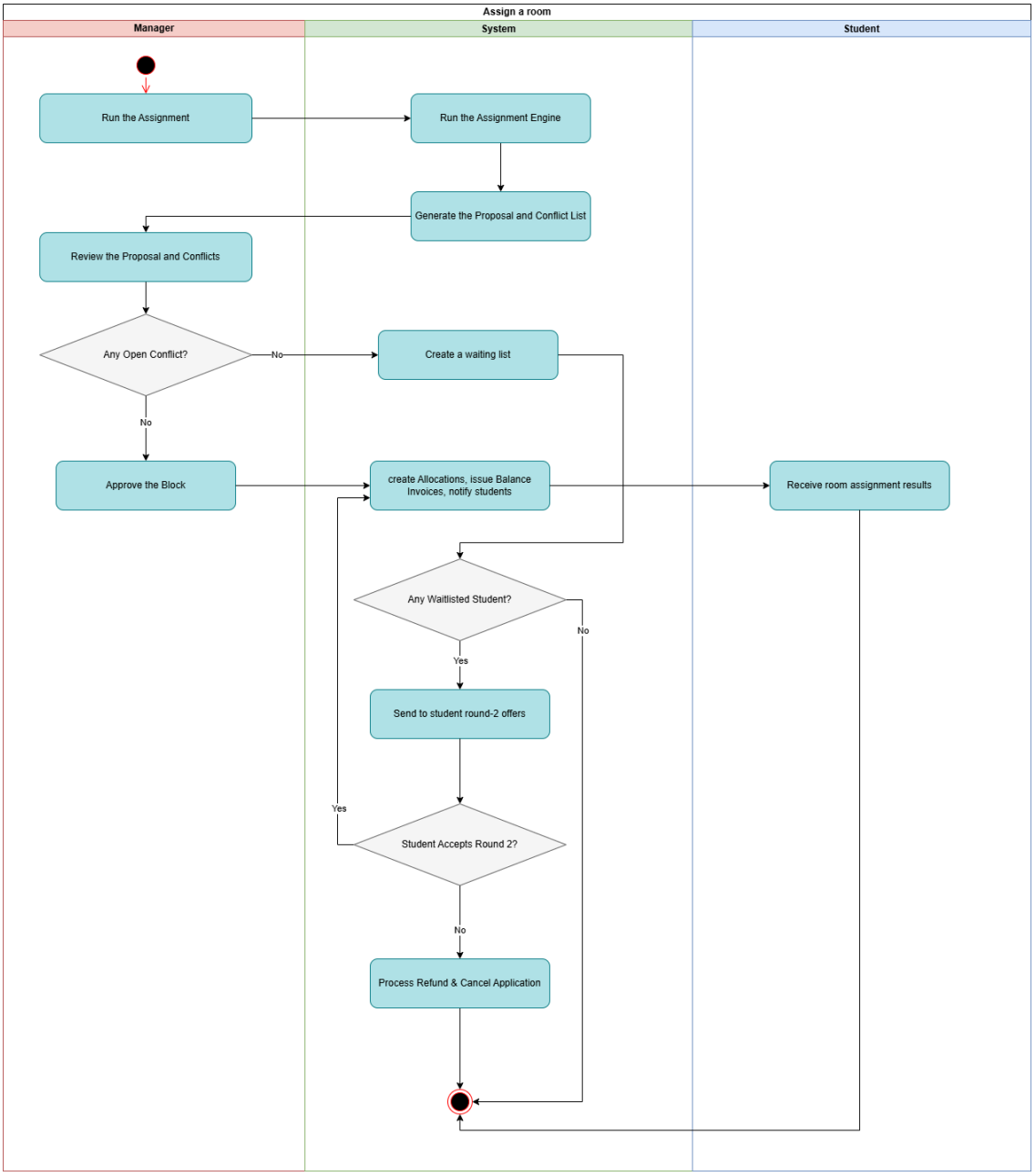


Figure 9: Activity diagram WF-02: Room & Bed Assignment

3.2.3 WF-03 Room Check-in Flow

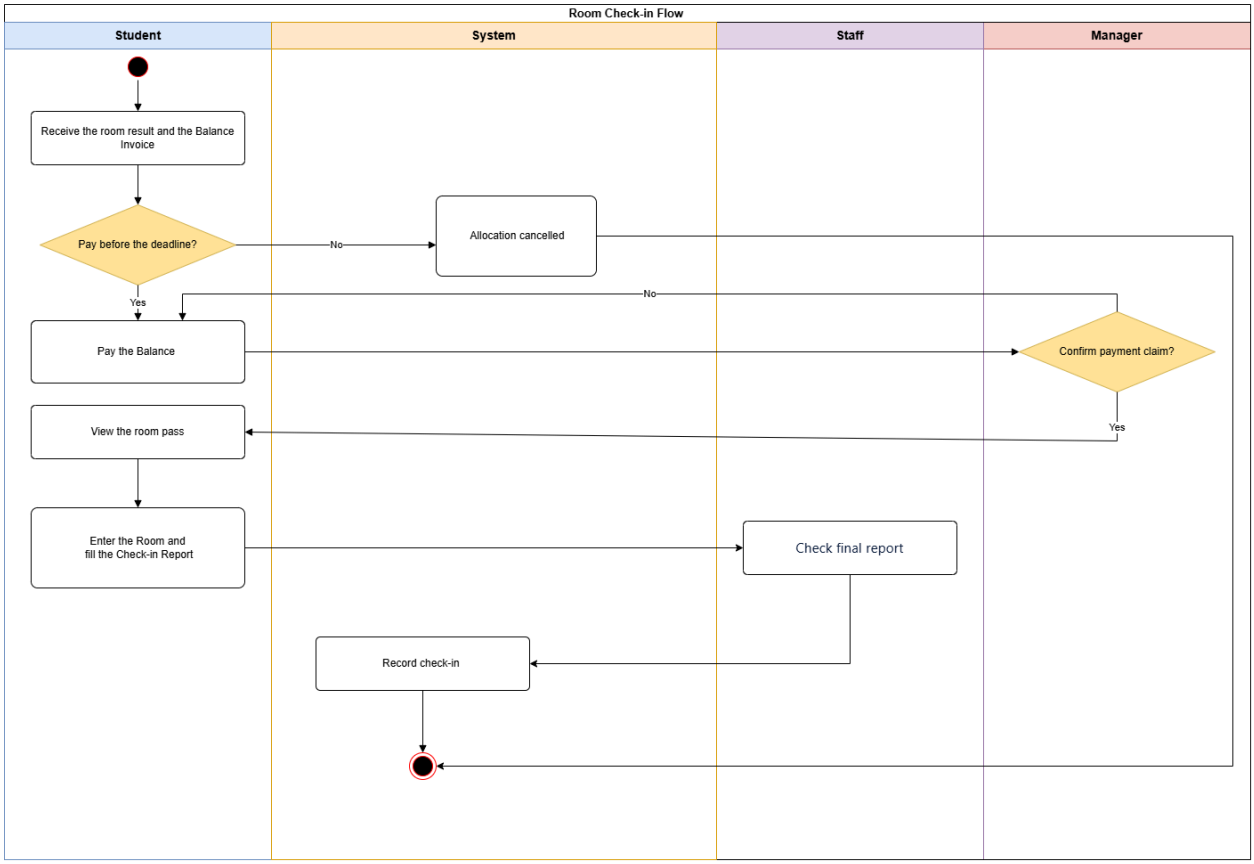


Figure 10: Activity diagram WF-03: Room Check-in Flow

3.2.4 WF-04 Mutual Room-Swap Request

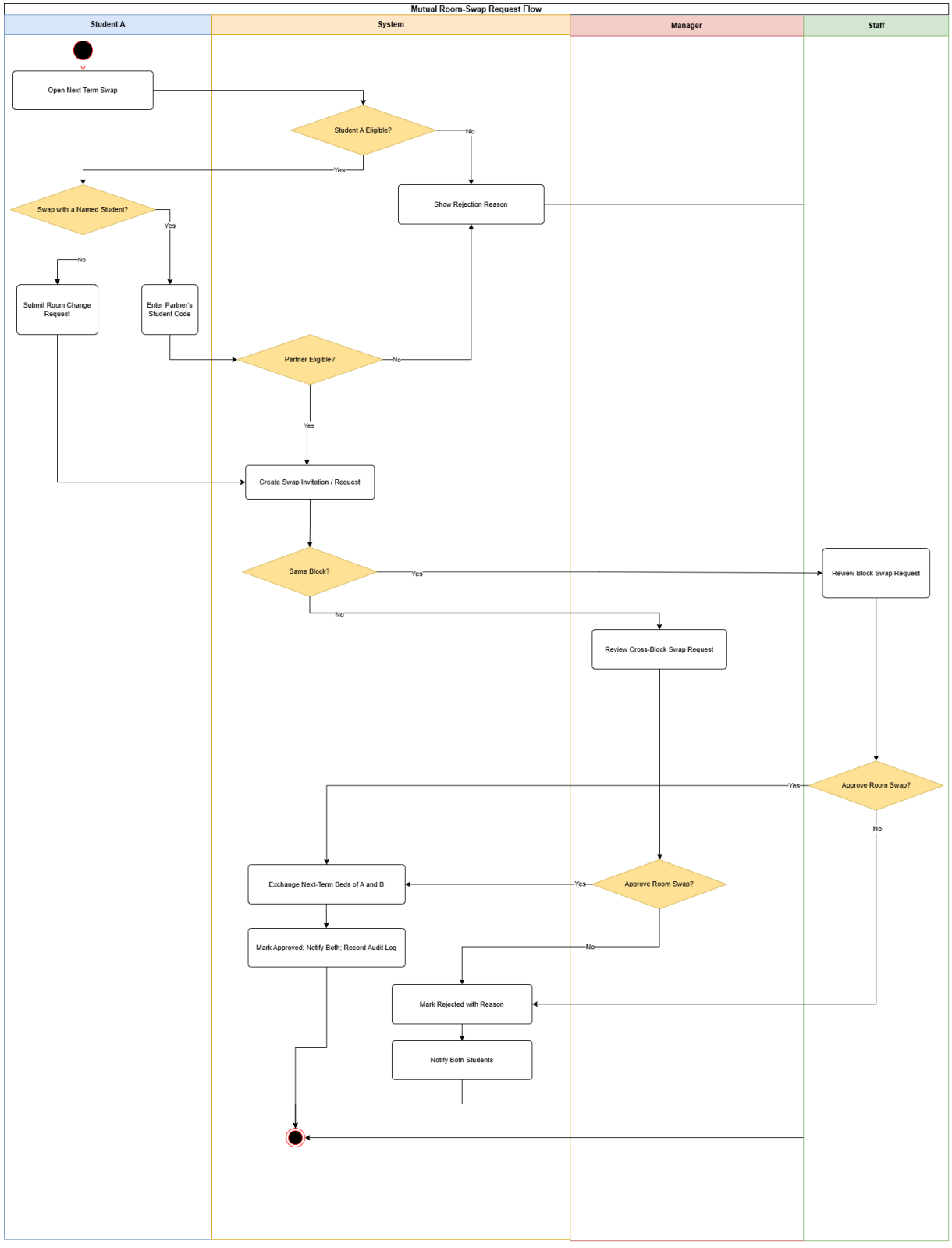


Figure 11: Activity diagram WF-04: Mutual Room-Swap Request

3.2.5 WF-05 Maintenance Request

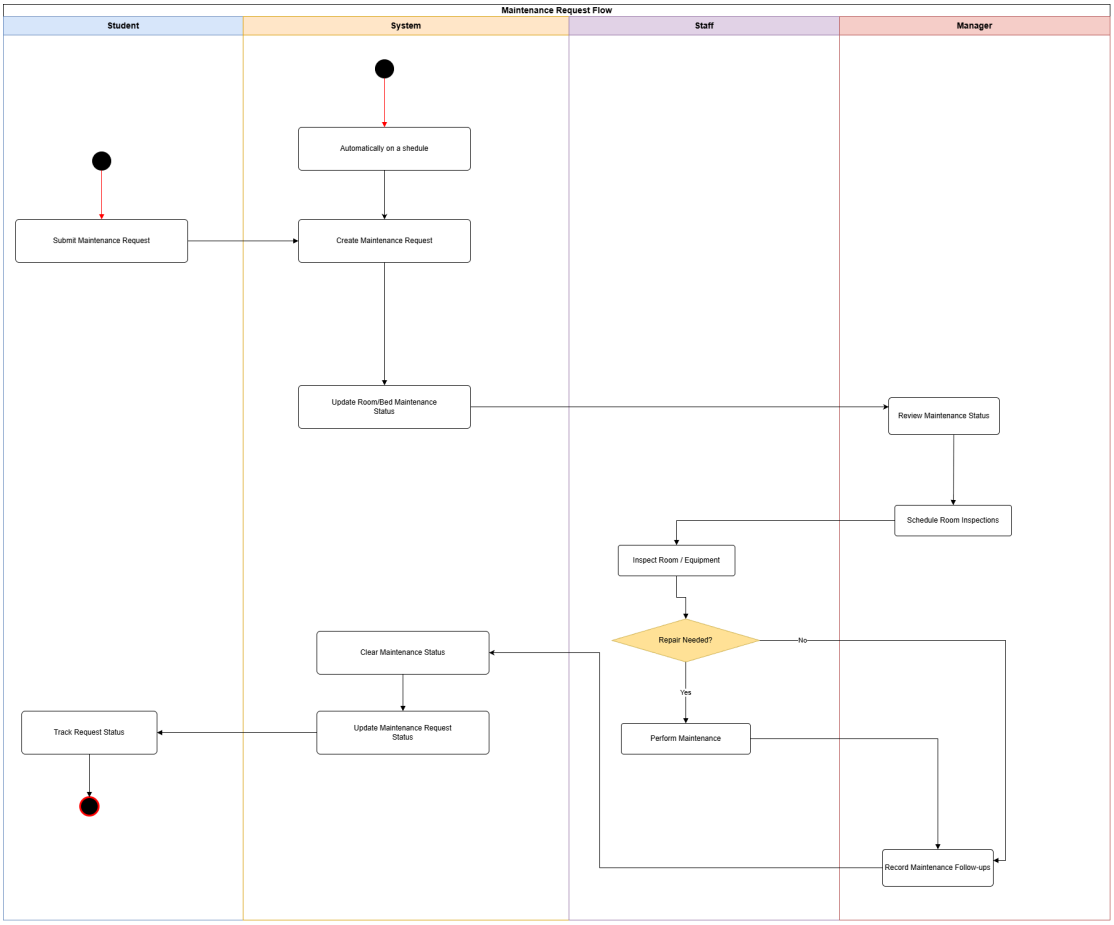


Figure 12: Activity diagram WF-05: Maintenance Request

3.3 Accounts, sign-in and sessions (AUTH)

Covers FR-AUTH-01…FR-AUTH-04. The same structure (controller → service → interface implemented in another feature, events for side effects) is used by every other feature; the following sections show only their own classes.

3.3.1 Class Diagram